SCYRIX

INDEPENDENT SECURITY RESEARCH

HOSTILE MINDS. SOVEREIGN HANDS.

SCYRIX.AI

Frisco, Texas

PUBLIC PORTFOLIO EDITION

BREAK IT.
OWN IT.
SECURE IT.

Representative Technical Work Sample

Assessment: Network-Connected Conferencing Device

Conducted December 19, 2025

SEC-01 SEC-02 SEC-03 SEC-04 SEC-05 SEC-06 SEC-07

PUBLICATION NOTE

Product and manufacturer identifiers, network identifiers, authentication artifacts, and personal contact details are gone from this edition. Referenced CVEs are withheld and implementation details are generalized to the device class. PoC and exploit source is replaced with same-footprint redaction; distinctive API paths, config keys, and protocol markers are placeholdered.

TECHNICAL REVIEW

The technical findings stand as written. Scoring, exploitability, and CVE-applicability statements await the researcher's final validation; this edition asserts no independent verification.

Website: scyrix.ai **Research: github.com/Scyrix-LLC**

DISCOVER	EXPLOIT	EMULATE	DETECT	AUTOMATE	VALIDATE

Assessment Report

Item	Detail
Target	Network-connected conference device (identifiers withheld)
Address	Withheld
Firmware	Withheld
Conducted	December 19, 2025

What We Found

Seven weaknesses. All confirmed. One high, five medium, one low.

7 CONFIRMED WEAKNESSES	**700+** TESTS EXECUTED	**315** XSS TECHNIQUES	**0** INJECTION BYPASSES

Scope caveat: none of this is reachable from the internet. Every finding begins from a position the attacker already holds, a seat on the internal network or a body within ten meters of Bluetooth range. Compromise the network first, or stand in the parking lot. Those are the doors.

The worst of it is a session that refuses to die, and even that waits on a token stolen out of the wire first. The crashes need local reach. The Bluetooth pairings need proximity. The crypto flaw needs a man-in-the-middle already in position.

The Seven Findings

#	Finding	Severity	Ease	Verified
1	Persistent Session Token Survives Logout	**HIGH**	Token theft required	**Confirmed**
2	Regex Exhaustion in Proxy Content Filtering	**MEDIUM**	Trivial	**Confirmed**
3	Numeric Boundary Crash in Config Parser	**MEDIUM**	Near-certain	**Confirmed**
4	Legacy Key Exposure Enables Offline Password Recovery	**MEDIUM**	Practical, cheap	**Confirmed**
5	USB Peripheral Tunnel over Bluetooth	**MEDIUM**	Demonstrated	**Confirmed**
6	Pairing Bypass via Legacy Modes	**MEDIUM**	Demonstrated	**Confirmed**
7	Client-Side Blank-Check Coercion	**LOW**	Blocked server-side	**Confirmed**

How the Device Holds Up

What held

- The input validation is genuinely good. Cross-site scripting, SQL injection, command injection: all bounced.
- Thirty-two obfuscated request-forgery payloads. Zero takers.
- Ten-megabyte payloads thrown at the parsers. No memory-safety failures.
- Configuration write endpoints check their callers.

- Parameter pollution: rejected.

What gave way

- Sessions never truly end.
- One crafted input pins a CPU core.
- One oversized integer takes the box down.
- 1024-bit RSA in this decade.

Infrastructure Already in Place

- Configuration files accept hardening, and provisioning can push it fleet-wide.
- Account lockout exists (the crash findings, SEC-02 and SEC-03, reset it with every restart).
- Central log aggregation already correlates logins, pairing events, and configuration changes across the fleet.

The pieces for fleet-wide defense are already on the table.

SEC-01 · Persistent Session Token Survives Logout

HIGH

Rated HIGH | CVSS 3.1: 8.0 (AV:A/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H) | Likelihood: Token theft required

> **Fastest fixes:** Trusted TLS certificates close the theft path | Authentication events to syslog review | Segment the management network

Overview

Logout is a suggestion. The server keeps the session alive after the user walks away, so a cookie stolen once keeps working indefinitely. The administrative interface never closes the door.

Mechanics

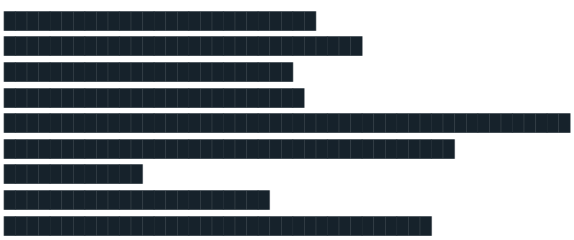
- Component: the session lifecycle layer
- Fault: logout answers success and leaves the session intact

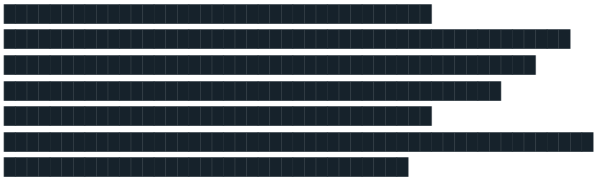
Sequence of events:

- A user signs in; a cookie is issued
- The cookie is intercepted or copied
- The user signs out
- The stolen cookie keeps working
- It works indefinitely; only timeout or reboot stops it

Demonstration

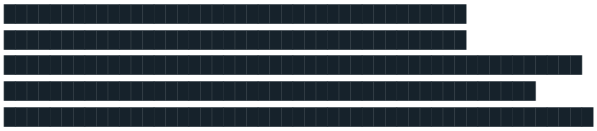
```
# [ SOURCE WITHHELD | Session persistence PoC | 16 lines ]
```





Observed Outcome

```
# [ SOURCE WITHHELD | Session test transcript | 5 lines ]
```



Consequences

- A captured cookie has no natural death.
- Seconds of physical access buy lasting administrative reach.
- Interception pays once and pays forever.
- Shared hardware inherits the previous user.
- Users cannot log out of an attack.

Plausible Scenarios

- The administrator logs out; the attacker's session does not.
- A cookie lifted from an unattended laptop, replayed at leisure.
- The conference room remembers the last user's session.
- One sniffed login funds weeks of access.

Fix Path

The firmware is frozen, so the fixes live upstream of it:

- Trusted TLS certificates kill the interception path.
- Segmentation walls off the administrative interface.
- Syslog review flags session reuse patterns.
- A reverse proxy can enforce a hard session ceiling.
- Layer on VPN, MFA, or source-IP allowlists.

SEC-02 · Regex Exhaustion in Proxy Content Filtering

MEDIUM

Rated MEDIUM | CVSS 3.1: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H) | Likelihood: Trivial

> **Fastest fixes:** A rate-limiting WAF in front (10 req/sec) | Request timeouts near 5 seconds | CPU alerts to syslog | Segmentation optional

> *Password guessing is throttled on the box, but a crash clears the throttle with every restart.*

Possible chain: this condition pairs neatly with password guessing.

- Spend a few guesses; the lockout trips.
- Fire the payload; the device restarts; the counter resets.

- Repeat.
- The wordlist lands eventually.
- Restart behavior alone defeats the lockout.

Overview

The proxy screens HTML with a regex that backtracks catastrophically on the wrong input. One unauthenticated request, one pinned core, an interface that stops answering.

Mechanics

- Component: the proxy's content screening stage
- Fault: the regex degenerates into exponential backtracking on adversarial markup
- Delivery: deep nesting, repeated unclosed tags, bloated attributes

Demonstration

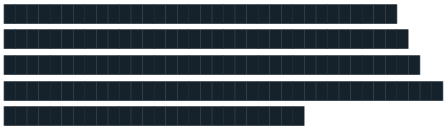
The payload:

```
# [ SOURCE WITHHELD | Regex exhaustion payload | 3 lines ]
```



Observed outcome:

```
# [ SOURCE WITHHELD | Timing measurements | 5 lines ]
```



Consequences

- The interface goes quiet.
- One request, one full core.
- No login required.
- A single command-line request does it.
- Seconds of CPU burned per message.

Plausible Scenarios

- Timed against the start of a critical meeting.
- Parallel requests occupy every core.
- A loop keeps the device down indefinitely.

The Guessing-Bypass Chain

The crash and the guessing pool their strengths.

Why the lockout loses:

- A handful of failures locks the account.
- The payload forces a restart.
- Restart wipes sessions and failure counters together.
- Crash-then-guess removes the ceiling.
- The wordlist gets worked through, batch by batch.

- The intended defense never engages.

```
# [ SOURCE WITHHELD | Guess-and-restart loop | 36 lines ]
```



What the attacker needs:

- Adjacent network access.
- A password that lives in a common wordlist.
- A minute per guess, restart included.
- 1,000 candidates resolve in roughly 16 unattended hours.

Fix Path

The firmware is frozen; the network answers instead:

- A WAF with rate limits and short timeouts.
- Filtering of malformed markup upstream.
- Syslog alerts on CPU spikes.
- Segmentation to shrink the surface.
- Upstream denial-of-service protection where availability matters.

SEC-03 · Numeric Boundary Crash in Config Parser

MEDIUM

Rated MEDIUM | CVSS 3.1: 7.5 (AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H) | Likelihood: Near-certain

Fastest fixes: Reject oversized integers at a WAF | Alert on unexpected restarts | Health checks that can power-cycle the box

Like SEC-02, every crash resets the lockout counter; guessing can continue forever.

Overview

Certain integers kill the application outright. The device goes silent and needs a reboot to come back.

Mechanics

- Component: numeric parsing and range checks
- Triggers: values at or past the 32-bit ceiling (2,147,483,647 up), the 64-bit ceiling (9,223,372,036,854,775,807), and the negative mirror (-2,147,483,649)
- Likely fault: a signed overflow in native code

Demonstration

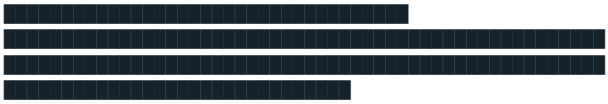
The payload:

```
# [ SOURCE WITHHELD | Boundary value payload | 4 lines ]
```



Observed outcome:

```
# [ SOURCE WITHHELD | Boundary test transcript | 4 lines ]
```



Consequences

- Total outage; a reboot is the only exit.
- Reachable without authentication (per earlier testing context).
- One JSON field is the whole weapon.
- Calls drop mid-sentence.

Plausible Scenarios

- Crash the device as the big call starts.
- Re-crash it the moment it recovers.
- Take any monitoring riding on the box down with it.
- Pair it with guessing, exactly as in the SEC-02 chain: each restart clears the lockout counter.

Fix Path

- A WAF that rejects out-of-range numbers.
- Syslog alerts on surprise restarts.

- Automated health checks with recovery teeth.
- Rate limiting against scripted re-crash loops.

SEC-04 · Legacy Key Exposure Enables Offline Password Recovery

MEDIUM

Rated MEDIUM | CVSS 3.1: 5.9 (AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N) | Likelihood: Practical, cheap

Fastest fixes: Trusted TLS certificates, above everything else: they break step two of the chain | Long random passwords per device | Segment away the interception window

Online guessing is throttled by lockout. This attack runs offline, against a captured ciphertext, where lockout means nothing.

Overview

The device hands its 1024-bit RSA public key to anyone who asks. Pair that with one intercepted login and passwords fall to offline guessing. Factoring the key never enters into it.

Mechanics

Three steps, no authentication on the first:

Step 1: the key walks out

```
# [ SOURCE WITHHELD | Key material retrieval | 7 lines ]
```

Step 2: the login gets intercepted (MITM, internal network)

```
# [ SOURCE WITHHELD | Credential interception | 5 lines ]
```

Step 3: the guessing runs offline

```
# [ SOURCE WITHHELD | Offline guessing loop | 8 lines ]
```

Why guessing wins here:

- One password, one key: always the same ciphertext.

- Testing a candidate needs only the public half.
- A match against the capture settles it.
- Short password ceilings keep the search small.

Consequences

Offline recovery, the primary threat:

- $0-50 on commodity hardware or a short cloud run.
- Minutes to hours, depending on password quality.
- High yield on short, dictionary-based passwords.
- Public tooling; low skill floor.

Attack economics:

Password shape	Candidate space	Time	Cost
Numeric (6 digits)	1 million	< 1 minute	$0
Alphanumeric (short)	2.8 trillion	2-8 hours	$10-50
Complex (short)	218 trillion	1-3 days	$50-200

Factoring the key (the hard road):

- $100k-1M: academic or nation-state money.
- Months of compute.
- Pointless while guessing is cheaper.

Existing Safeguards

Working in the defender's favor:

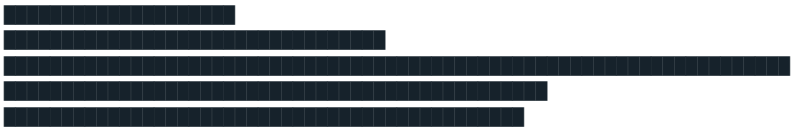
- HTTPS wraps everything a second time.
- The key rotates each session.
- Ciphertexts live for seconds.
- HTTPS has to fall first.

What still bites:

- Break HTTPS and the 1024-bit key falls next.
- The key size is behind the times, full stop.
- PCI-DSS and FIPS 140-2 both reject it.

Demonstration

```
# [ SOURCE WITHHELD | Key size probe | 5 lines ]
```



Fix Path

- Trusted certificates remove step two entirely.
- Twelve-plus random characters push the search out of reach.
- Segmentation narrows the interception window.

Certificates plus strong passwords make the exposed key an academic curiosity.

SEC-05 · USB Peripheral Tunnel over Bluetooth MEDIUM

Rated MEDIUM | CVSS 3.1: 6.5 (AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N) | Likelihood: Demonstrated

Fastest fixes: Bluetooth off when unused (the PIN rotates on its own and cannot be set) | Pairing events to syslog | Physical-access controls

Overview

A publicly disclosed USB-over-Bluetooth flaw (CVE withheld) sits in the stack. Pair once, and a remote system can open a USB channel to the device, reaching peripherals and their data, with no further authentication.

Mechanics

- Reference: publicly disclosed, CVE identifier withheld
- Entry: a completed pairing, then the USB-over-Bluetooth protocol
- Gate: the pairing, and nothing after it
- Consequence: remote reach into USB peripherals
- Established by: Bluetooth protocol analysis
- Disclosure: publicly documented

What the Channel Allows

After pairing, the channel lets the far end:

- Enumerate attached peripherals.
- Open channels to them.
- Move data to and from USB storage.
- In principle, issue commands to USB-connected hosts.

Exploitation sequence:

- Pair once, in physical range.
- Pairing buys trust.
- The USB-over-Bluetooth channel opens.
- Peripheral access follows, unauthenticated.
- It persists at Bluetooth range afterward.

The Syscall Filter Changes the Picture

The device runs SECCOMP, and the filter takes most of the teeth out of this.

What the filter permits:

- USB works only as recording storage.
- Meeting recording to USB media is the sanctioned use.
- Every use passes an on-screen consent prompt.
- No arbitrary execution on the host.
- No camera or microphone beyond the sanctioned path.

- No USB network adapters.

Effect of the restriction:

```
# [ SOURCE WITHHELD | Impact comparison | 2 lines ]
```



Realistic exploitation, filter in place:

- The attacker pairs from physical range.
- USB access is attempted.
- The filter cuts it down to storage.
- The consent prompt appears.
- A decline ends it.
- An acceptance buys stored recordings, nothing more.

What remains: even with consent granted, the attacker never gets:

- Live camera or microphone feeds.
- Arbitrary execution.
- Other peripheral classes (network adapters, keyboards, and so on).

Practical Impact

Affected scenarios, filter-limited:

- USB storage holding recorded meetings, consent required.
- Cameras and microphones: filtered out.
- Network adapters: filtered out.
- Debug interfaces: filtered out.

Capabilities, reduced by the filter:

- Exfiltration: storage only, consent-gated.
- Eavesdropping: blocked.
- Lateral movement: blocked.
- Persistence: the pairing survives; the exploitation stays bounded.

Plausible Settings

Corporate: pair after hours, return from the parking lot. Direct peripheral control is filtered away; what is left is recorded meetings, and only if someone accepted the prompt. History, not live video.

Hotel rooms: a previous guest's pairing cannot watch later meetings. It can reach recordings that exist because someone accepted the prompt. Archives, not eavesdropping.

Fix Path

> *The pairing PIN rotates automatically and cannot be set.*

- Bluetooth off when it serves no purpose.
- Pairing and USB events to syslog.
- Control the ten meters around the device.
- Notice who lingers.

SEC-06 · Pairing Bypass via Legacy Modes

MEDIUM

Rated MEDIUM | CVSS 3.1: 5.7 (AV:A/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N) | Likelihood: Demonstrated

Fastest fixes: Bluetooth off when unused (the PIN rotates on its own and cannot be set) | Alert on unexplained pairing attempts | Physical-access controls

Overview

A publicly disclosed improper-authentication flaw (CVE withheld) theoretically lets an attacker skip the pairing handshake. In practice the device stacks four controls against it.

Mechanics

- Reference: publicly disclosed, CVE identifier withheld
- Entry: a weakness in the pairing handshake
- Gate: none, or a weak PIN; the handshake can be sidestepped
- Reach: roughly 10 meters (Class 2 radio)
- Consequence: unearned device trust; the on-ramp into SEC-05
- Disclosure: publicly documented across multiple Bluetooth stacks

Controls That Blunt the Impact

Four controls, stacked:

1. The screen talks. Every pairing attempt shows in real time. Not silent. Anyone in the room can refuse it.

2. Consent still gates the payload. A successful pair only leads to the on-screen recording prompt, which cannot be skipped. Storage access is never automatic.

3. The syscall filter applies. Whatever exploits this weakness inherits the same SECCOMP ceiling as SEC-05: storage-only recording, no execution, no camera, no adapters.

4. Rate limiting throttles the loop. Automated pairing attempts get slowed or blocked outright.

What exploitation actually requires:

- A body within ten meters.
- A pairing that shows up on screen.
- A user who accepts the storage prompt.
- Even then: recordings only, per the filter.
- Automation denied by throttling.

Why Pairing Is Weak

The handshake has three structural problems.

1. A PIN that rotates: short, regenerated at every boot, unsettable by users, optional in Just Works mode, and never locked out on failure.

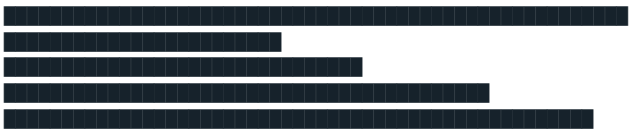
2. Legacy modes: Secure Simple Pairing Just Works is accepted, confirmation is skipped, and MitM exposure comes with the design.

3. No allowlist: anyone may initiate, no peer verification, pairings persist.

Approaches

Approach 1: forcing Just Works (the primary route)

```
# [ SOURCE WITHHELD | Legacy pairing commands | 5 lines ]
```



Approach 2: guessing the PIN (rarely practical)

```
# [ SOURCE WITHHELD | PIN guessing loop | 15 lines ]
```



Net Effect

Direct risks, already blunted by the controls above:

- Pairing status for the asking (visibly).
- The on-ramp into SEC-05 (still consent-gated).
- Pairings that survive reboots (still visible, still removable).

Scenarios, with controls in mind

1. Social engineering: the device impersonates a peripheral; the screen shows the pairing; success needs two moments of user inattention, one at pairing and one at the prompt. Either refusal ends it.

2. Proximity: after-hours automation from a neighboring space. Throttling slows it, the screen betrays it, the prompt still waits.

3. Supply chain: a pre-paired device shows up in settings and gets removed. The prompt is the last gate.

Combined attack (SEC-06 + SEC-05)

The two chain in theory; the stack of controls keeps the outcome narrow:

- [SEC-06] Pairing attempt.
- Visible on screen.
- User accepts (first yes).
- [SEC-05] USB access attempt.
- User accepts storage (second yes).
- Both yeses: filtered access to stored recordings.
- Net: archives only. No live feed, no execution.

Where the defenses stack up:

- Pairing is loud, never silent.
- Two consents gate the chain.
- Throttling denies automation.
- The filter caps the damage.
- Bottom line: recorded meetings, at best, after two yeses.

Fix Path

Mirrors SEC-05:

- Bluetooth off when unneeded.
- Pairing attempts to syslog.
- Control the ten meters.

> *The PIN is short, rotates on its own, and cannot be set. Throttling already answers automation.*

SEC-07 · Client-Side Blank-Check Coercion

LOW

Rated LOW | CVSS 3.1: 3.1 (AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N) | Likelihood: Blocked server-side

Fastest fixes: Nothing to do. The server already refuses every attempt. This is a code-quality note, not a security finding.

Overview

The client ships a blank-value check built on loose coercion: the number 0 and the boolean false both read as empty. A validation quirk, not a hole.

Mechanics

- Component: the deobfuscated client bundle

Vulnerable code:

```
# [ SOURCE WITHHELD | Vendor validation function | 3 lines ]

[redacted]
```

Coercion behavior:

```
# [ SOURCE WITHHELD | Coercion behavior transcript | 5 lines ]

[redacted]
```

Consequences

In theory:

- A modified client, or a raw 0 or false where a value belongs, slips the client-side gate.

In practice:

- 36 type-confusion payloads submitted.

- Every one answered 403 or a validation error.
- The server re-checks everything the client claims.

Demonstration

```
# [ SOURCE WITHHELD | Type confusion probe | 5 lines ]
```



Observed Outcome

Across 36 probes, every write endpoint returned 403. No bypass. The server simply does not trust the client.

Fix Path

The client defect stands; the server already pays for it. No action warranted.

> *The bundle is minified and unchangeable without firmware access. The server's answer is final: all 36 payloads blocked.*

How It Was Tested

What Was Covered

Test area	Volume	Outcome
Session handling	5	1 high
Availability and crash behavior	3	2 medium
Cryptographic configuration	1	1 medium
Input validation (all classes)	700+ tests	0 weaknesses
- Cross-site scripting	315 payloads, 35 techniques	**Secure**
- Server-side request forgery	32 obfuscated payloads	**Secure**
- SQL injection	multiple payloads	**Secure**
- Command injection	multiple payloads	**Secure**
- Memory safety	payloads up to 10 MB	**Secure**
- Parameter pollution	24 tests	**Secure**
- Prototype pollution	multiple attempts	**Secure**
- Path traversal	multiple attempts	**Secure**
- XML external entities	multiple attempts	**Secure**
- Format strings	multiple attempts	**Secure**

Artifacts Produced

Every script, PoC, fuzzer, and tool built for this assessment went into the coordinated disclosure package. None of it appears in this edition.

Recommended Actions

> *Every finding assumes internal-network access or physical proximity. Configuration and network hardening carry the day.*

Do These First

1. Trusted TLS certificates. They close the interception path behind SEC-01 and SEC-04.

2. Strong passwords. Twelve-plus random characters per device blunts SEC-01 and SEC-04.

3. Bluetooth off, unless the mission needs it. That retires SEC-05 and SEC-06. (The PIN rotates on its own and cannot be set.)

4. Central logging. Failed logins, pairing events, surprise restarts: one place.

Worth Considering

5. Segmentation. A dedicated management VLAN for the fleet.

6. A WAF. Rate limits and input validation where availability matters; answers SEC-02 and SEC-03. (Device-level rate limiting already exists.)

Overall Assessment

Against the classic web-attack playbook, the conference device is a hard target: 700+ tests, zero XSS, zero SQL injection, zero request forgery, zero memory-safety failures. Someone cared about input validation.

Key point: every finding starts from a foothold the attacker already holds, the internal network or ten meters of proximity. Nothing here is internet-reachable.

Most serious: the immortal session (SEC-01), and even it waits on a stolen token. Certificates and strong passwords close it.

The crashes: local reach required; rate limiting contains them.

The crypto: obsolete at 1024 bits, and irrelevant behind trusted certificates.

Overall risk: MEDIUM

Configured properly (trusted certificates, strong passwords, Bluetooth off), the device is production ready. Every finding has a configuration-level answer. No firmware required.

Appendix A: Exposure Summary

Vector	Reach	Authentication	Effect	Difficulty
Session replay	Network / physical	No (cookie theft)	Full administrative compromise	Easy

Vector	Reach	Authentication	Effect	Difficulty
Regex exhaustion	Network	No	Service outage	Easy
Boundary value crash	Network	No*	Device crash	Easy
Offline password recovery	Network	No	Credential disclosure	Very hard
Transport interception	Network	No	Information disclosure	Medium

**Varies by endpoint*

Appendix B: Scoring Detail

Finding	CVSS v3.1 Vector	Score
SEC-01	CVSS:3.1/AV:A/AC:L/PR:N/UI:R/S:U/C:H/I:H/A:H	**8.0 (High)**
SEC-02	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H	**7.5 (Medium)**
SEC-03	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H	**7.5 (Medium)**
SEC-04	CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:N/A:N	**5.9 (Medium)**
SEC-05	CVSS:3.1/AV:A/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	**6.5 (Medium)**
SEC-06	CVSS:3.1/AV:A/AC:L/PR:N/UI:R/S:U/C:N/I:H/A:N	**5.7 (Medium)**
SEC-07	CVSS:3.1/AV:N/AC:H/PR:N/UI:R/S:U/C:N/I:L/A:N	**3.1 (Low)**

Edition 1.0 | Prepared December 19, 2025 | Revisit after remediation